

Relatório – Sistema de Autocomplete de Jogos com Trie

Introdução

O objetivo deste projeto foi desenvolver um sistema de autocomplete para títulos de jogos utilizando a estrutura de dados Trie. Esse tipo de estrutura é amplamente utilizado em mecanismos de busca, corretores ortográficos e sistemas de sugestão automática, pois permite localizar palavras a partir de prefixos de maneira muito eficiente. O projeto utiliza uma base de jogos já fornecida e implementa operações de inserção, busca exata e autocomplete, retornando os resultados mais relevantes de acordo com a popularidade dos jogos.

1. Representação da Trie

A estrutura principal utilizada foi uma Trie (árvore de prefixos). Cada nó da Trie é representado pela classe `TrieNode`, que contém um vetor de ponteiros para os filhos, um indicador booleano chamado `isEndOfTitle` e um ponteiro para um objeto `Game`. O indicador booleano informa se o caminho percorrido até aquele nó representa um título completo armazenado na estrutura.

Foi adotado um alfabeto de tamanho 36, composto pelas 26 letras do alfabeto inglês e pelos 10 dígitos numéricos. A função `charToIndex()` realiza a conversão de cada caractere para uma posição específica do vetor de filhos. Essa abordagem permite acesso direto aos nós filhos em tempo constante.

Para tornar as buscas mais robustas, foi implementada a função `toSearchKey()`. Essa função converte todos os caracteres para letras minúsculas, remove espaços em branco e mantém apenas letras e números válidos. Dessa forma, títulos como “Half Life”, “half life” e “HALF LIFE” são tratados de maneira equivalente durante as consultas.

2. Funcionamento dos Métodos

Método `insert()`

O método `insert` recebe um ponteiro para um objeto `Game`. Inicialmente, o título é convertido para a chave de busca através de `toSearchKey()`. Em seguida, cada caractere da chave é percorrido. Caso um nó correspondente ainda não exista, ele é criado dinamicamente. Ao final do processo, o nó final é marcado como término de um título válido e recebe o ponteiro para o jogo associado.

Essa implementação evita a criação de cópias desnecessárias dos objetos `Game`, economizando memória e mantendo apenas referências aos elementos já existentes na base de dados.

Método `contains()`

O método `contains` verifica se um determinado título existe na Trie. Após converter o texto informado para a chave de busca, o algoritmo percorre a estrutura seguindo os caracteres da chave. Caso algum caminho não exista, o método retorna falso imediatamente. Se o percurso for concluído, o resultado dependerá do valor de `isEndOfTitle` no último nó visitado.

Essa verificação garante que apenas títulos completos sejam reconhecidos como válidos, evitando que prefixos parciais sejam considerados jogos cadastrados.

Método `autocomplete()`

O método `autocomplete` é responsável por retornar sugestões de jogos a partir de um prefixo. Primeiramente, o algoritmo localiza o nó correspondente ao prefixo informado. Em seguida, realiza uma busca em profundidade (DFS) utilizando o método auxiliar `collectAll()`, que percorre toda a subárvore associada ao prefixo e coleta os jogos encontrados.

Após a coleta, os resultados são ordenados por popularidade em ordem decrescente. Quando dois jogos possuem a mesma popularidade, é utilizada a ordem alfabética da chave de busca como critério de desempate. Por fim, apenas os k primeiros resultados são retornados ao usuário.

3. Análise de Complexidade

Considere L como o tamanho da chave do título, p como o tamanho do prefixo, s como a

quantidade de nós visitados na subárvore e r como o número de resultados encontrados.

Inserção: $O(L)$. O algoritmo percorre cada caractere do título apenas uma vez.

Busca exata: $O(L)$. O custo é proporcional ao tamanho da chave pesquisada.

Autocomplete: $O(p + s + r \log r)$. O primeiro termo corresponde à localização do prefixo, o segundo à exploração da subárvore e o terceiro à ordenação dos resultados encontrados.

Na prática, a operação de autocomplete apresenta excelente desempenho porque a busca ocorre apenas na região da Trie relacionada ao prefixo informado, evitando percorrer toda a base de dados.

4. Comparação com uma Solução Ingênu

Uma solução simples poderia armazenar todos os jogos em um vetor e, para cada consulta, percorrer toda a lista verificando quais títulos começam com o prefixo desejado. Considerando n jogos cadastrados, o custo seria aproximadamente $O(n \cdot p)$, já que cada elemento precisaria ser analisado individualmente.

A Trie elimina a necessidade de verificar todos os títulos. Após localizar o nó correspondente ao prefixo, apenas a subárvore relevante é explorada. Isso torna a busca significativamente mais rápida em bases grandes, especialmente quando existem milhares de títulos cadastrados.

5. Uso de Memória e Considerações Finais

Embora a Trie apresente vantagens importantes em desempenho, ela exige mais memória do que estruturas lineares. Cada nó mantém um vetor fixo de 36 ponteiros, além de informações adicionais. Entretanto, esse custo é parcialmente compensado pelo compartilhamento de prefixos entre diferentes títulos. Jogos que possuem sequências iniciais semelhantes utilizam os mesmos nós da árvore, reduzindo a redundância de armazenamento.

Conclui-se que a Trie é uma estrutura adequada para sistemas de autocomplete devido à sua eficiência na pesquisa por prefixos. O projeto implementado atende aos requisitos propostos, oferecendo inserção eficiente, busca exata de títulos e geração de sugestões ordenadas por relevância. Além disso, a normalização das chaves de busca torna o sistema mais amigável ao usuário, permitindo consultas independentes de letras maiúsculas, minúsculas ou espaços em branco.